Creating maps and visualizing Geospatial data

Estimated time needed: **30** minutes

Objectives

After completing this lab you will be able to:

- Create maps and visualize geospatial data with Folium

Introduction

In this lab, we will learn how to create maps for different objectives. To do that, we will part ways with Matplotlib and work with another Python visualization library, namely **Folium**. What is nice about **Folium** is that it was developed for the sole purpose of visualizing geospatial data. While other libraries are available to visualize geospatial data, such as **plotly**, they might have a cap on how many API calls you can make within a defined time frame. **Folium**, on the other hand, is completely free.

Table of Contents

Exploring Datasets

Toolkits: This lab heavily relies on *pandas* and *Numpy* for data wrangling, analysis, and visualization. The primary plotting library we will explore in this lab is **Folium**.

Datasets:

1. San Francisco Police Department Incidents for the year 2016 - Police Department Incidents from San Francisco public data portal. Incidents derived from San Francisco Police Department (SFPD) Crime Incident Reporting system. Updated daily, showing data for the entire year of 2016. Address and location has been anonymized by moving to mid-block or to an intersection. Note: this dataset no longer exists on the original website since systems updates in the department. The link included will take you to the page explaining the change of system since this exercise was created.

2. Immigration to Canada from 1980 to 2013 - International migration flows to and from selected countries - The 2015 revision from United Nation's website. The dataset contains annual data on the flows of international migrants as recorded by the countries of destination. The data presents both inflows and outflows according to the place of birth, citizenship or place of previous / next residence both for foreigners and nationals. For this lesson, we will focus on the Canadian Immigration data and use the *already cleaned dataset*.

   You can refer to the lab on data pre-processing wherein this dataset is cleaned for a quick refresh your Pandas skill Data pre-processing with Pandas

Importing Libraries

Import Primary Modules:

In [1]:
```
import numpy as np
import pandas as pd
import folium
from branca.element import Figure
```

Introduction to Folium

Folium is a powerful Python library that helps you create several types of Leaflet maps. The fact that the Folium results are interactive makes this library very useful for dashboard building.

From the official Folium documentation page:

> Folium builds on the data wrangling strengths of the Python ecosystem and the mapping strengths of the Leaflet.js library. Manipulate your data in Python, then visualize it in on a Leaflet map via Folium.

> Folium makes it easy to visualize data that's been manipulated in Python on an interactive Leaflet map. It enables both the binding of data to a map for choropleth visualizations as well as passing Vincent/Vega visualizations as markers on the map.

> The library has a number of built-in tilesets from OpenStreetMap, Mapbox, Cartodb and supports custom tilesets with Mapbox or Cloudmade API keys. Folium supports both GeoJSON and TopoJSON overlays, as well as the binding of data to those overlays to create choropleth maps with color-brewer color schemes.

Generating the world map is straightforward in **Folium**. You simply create a **Folium** *Map* object, and then you display it. What is attractive about **Folium** maps is that they are interactive, so you can zoom into any region of interest despite the initial zoom level.

In [2]:
```
# define the world map
"""fig = Figure(800, 400)
world_map = folium.Map(zoom_start=4)"""

# display world map
"""fig.add_child(world_map)"""
```

Out[2]:
```
'fig.add_child(world_map)'
```

Go ahead. Try zooming in and out of the rendered map above.

You can customize this default definition of the world map by specifying the centre of your map, and the initial zoom level.

All locations on a map are defined by their respective *Latitude* and *Longitude* values. So you can create a map and pass in a center of *Latitude* and *Longitude* values of **[0, 0]**.

For a defined center, you can also define the initial zoom level into that location when the map is rendered. **The higher the zoom level the more the map is zoomed into the center**.

Let's create a map centered around Canada and play with the zoom level to see how it affects the rendered map.

In [3]:
```
# define the world map centered around Canada with a low zoom level
"""fig1 = Figure(800, 400)
world_map = folium.Map(location=[56.130, -106.35], zoom_start=4)"""

# display world map
"""fig1.add_child(world_map)"""
```

Out[3]:
```
'fig1.add_child(world_map)'
```

Let's create the map again with a higher zoom level.

In [4]:
```
# define the world map centered around Canada with a higher zoom level
"""fig2 = Figure(800, 400)
world_map = folium.Map(location=[56.130, -106.35], zoom_start=8)"""

# display world map
"""fig2.add_child(world_map)"""
```

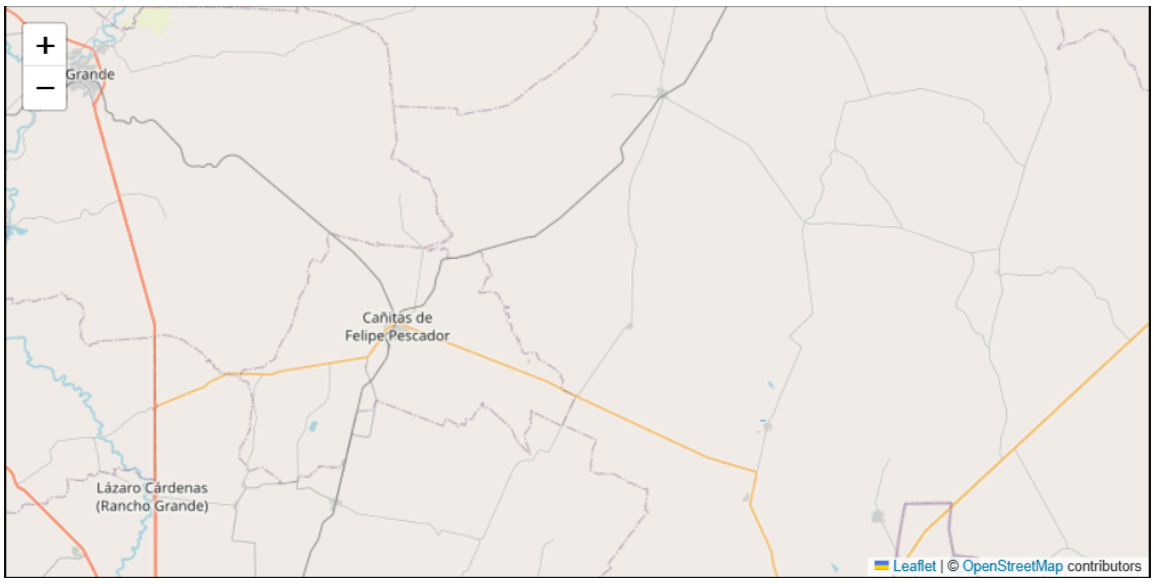
Out[4]:
```
'fig2.add_child(world_map)'
```

As you can see, the higher the zoom level the more the map is zoomed into the given center.

Question: Create a map of Mexico with a zoom level of 4.

In [5]:
```
### type your answer here
"""fig3 = Figure(800, 400)
mexico_latitude = 23.6345
mexico_longitude = -102.5528
mexico = folium.Map(location = [mexico_latitude, mexico_longitude])
fig3.add_child(mexico)"""
```

Out[5]:
```
'fig3 = Figure(800, 400)\nmexico_latitude = 23.6345 \nmexico_longitude = -102.5528
\nmexico = folium.Map(location = [mexico_latitude, mexico_longitude])\nfig3.add_ch
ild(mexico)'
```



Another cool feature of **Folium** is that you can generate different map styles.

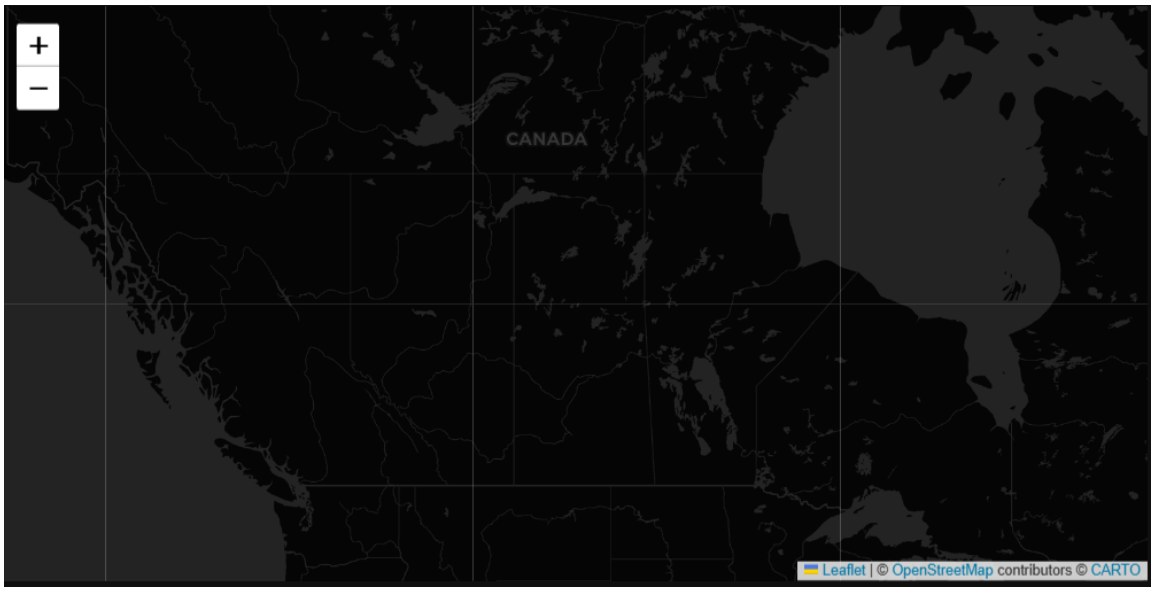
A. Cartodb dark_matter Maps

These are high-contrast B+W (black and white) maps. They are perfect for data mashups and exploring river meanders and coastal zones.

Let's create a Cartodb dark_matter map of canada with a zoom level of 4.

In [6]:
```
# create a Cartodb dark_matter map of the world centered around Canada
"""fig4 = Figure(800, 400)
world_map = folium.Map(location=[56.130, -106.35], zoom_start=4, tiles='Cartodb dar

# display map
fig4.add_child(world_map)"""
```

Out[6]:
```
"fig4 = Figure(800, 400)\nworld_map = folium.Map(location=[56.130, -106.35], zoom_
start=4, tiles='Cartodb dark_matter')\n\n# display map\nfig4.add_child(world_map)"
```



Feel free to zoom in and out to see how this style compares to the default one.

B. Cartodb positron Maps

CartoDB Positron maps are designed with a light and minimalistic aesthetic. They have a white or light-colored background and feature simple, clean lines for map elements. These maps are known for their modern and visually appealing design.

Let's create a Cartodb positron map of Canada with zoom level 4.

In [7]:
```
# create a Cartodb positron map of the world centered around Canada
"""fig5 = Figure(800, 400)
world_map = folium.Map(location=[56.130, -106.35], zoom_start=4, tiles='Cartodb pos

# display map
"""fig5.add_child(world_map)"""
```

Out[7]:
```
'fig5.add_child(world_map)'
```

Feel free to zoom in and out to see how this style compares to Cartodb dark_matter, and the default style.

Zoom in and notice how the borders start showing as you zoom in, and the displayed country names are in English.

Maps with Markers

Let's download and import the data on police department incidents using *pandas* `read_csv()` method.

Download the dataset and read it into a *pandas* dataframe:

In [8]:
```
df_incidents = pd.read_csv('Police_Department_Incidents_2016.csv')
```

Let's take a look at the first five items in our dataset.

In [9]:
```
df_incidents.head()
```

Out[9]:

	IncidntNum	Category	Descript	DayOfWeek	Date	Time	PdDistrict	Res
0	120058272	WEAPON LAWS	POSS OF PROHIBITED WEAPON	Friday	01/29/2016 12:00:00 AM	11:00	SOUTHERN	B
1	120058272	WEAPON LAWS	FIREARM, LOADED, IN VEHICLE, POSSESSION OR USE	Friday	01/29/2016 12:00:00 AM	11:00	SOUTHERN	B
2	141059263	WARRANTS	WARRANT ARREST	Monday	04/25/2016 12:00:00 AM	14:59	BAYVIEW	B
3	160013662	NON-CRIMINAL	LOST PROPERTY	Tuesday	01/05/2016 12:00:00 AM	23:50	TENDERLOIN	
4	160002740	NON-CRIMINAL	LOST PROPERTY	Friday	01/01/2016 12:00:00 AM	00:30	MISSION	

So each row consists of 13 features:

> 1. **IncidntNum**: Incident Number
> 2. **Category**: Category of crime or incident
> 3. **Descript**: Description of the crime or incident
> 4. **DayOfWeek**: The day of week on which the incident occurred

> 5. **Date**: The Date on which the incident occurred
> 6. **Time**: The time of day on which the incident occurred
> 7. **PdDistrict**: The police department district
> 8. **Resolution**: The resolution of the crime in terms whether the perpetrator was arrested or not
> 9. **Address**: The closest address to where the incident took place
> 10. **X**: The longitude value of the crime location
> 11. **Y**: The latitude value of the crime location
> 12. **Location**: A tuple of the latitude and the longitude values
> 13. **PdId**: The police department ID

Let's find out how many entries there are in our dataset.

In [10]:
```
df_incidents.shape
```

Out[10]:
```
(150500, 13)
```

So the dataframe consists of 150,500 crimes, which took place in the year 2016. In order to reduce computational cost, let's just work with the first 100 incidents in this dataset.

In [11]:
```
# get the first 100 crimes in the df_incidents dataframe
limit = 100
df_incidents = df_incidents.iloc[0:limit, :]
```

Let's confirm that our dataframe now consists only of 100 crimes.

In [12]:
```
df_incidents.shape
```

Out[12]:
```
(100, 13)
```

Now that we reduced the data a little, let's visualize where these crimes took place in the city of San Francisco. We will use the default style, and we will initialize the zoom level to 12.

In [13]:
```
# San Francisco latitude and longitude values
latitude = 37.77
longitude = -122.42
```

In [14]:
```
# create map and display it
"""fig6 = Figure(800, 400)
sanfran_map = folium.Map(location=[latitude, longitude], zoom_start=12)"""

# display the map of San Francisco
"""fig6.add_child(sanfran_map)"""
```

Out[14]:
```
'fig6.add_child(sanfran_map)'
```

Now let's superimpose the locations of the crimes onto the map. The way to do that in **Folium** is to create a *feature group* with its own features and style and then add it to the

`sanfran_map`.

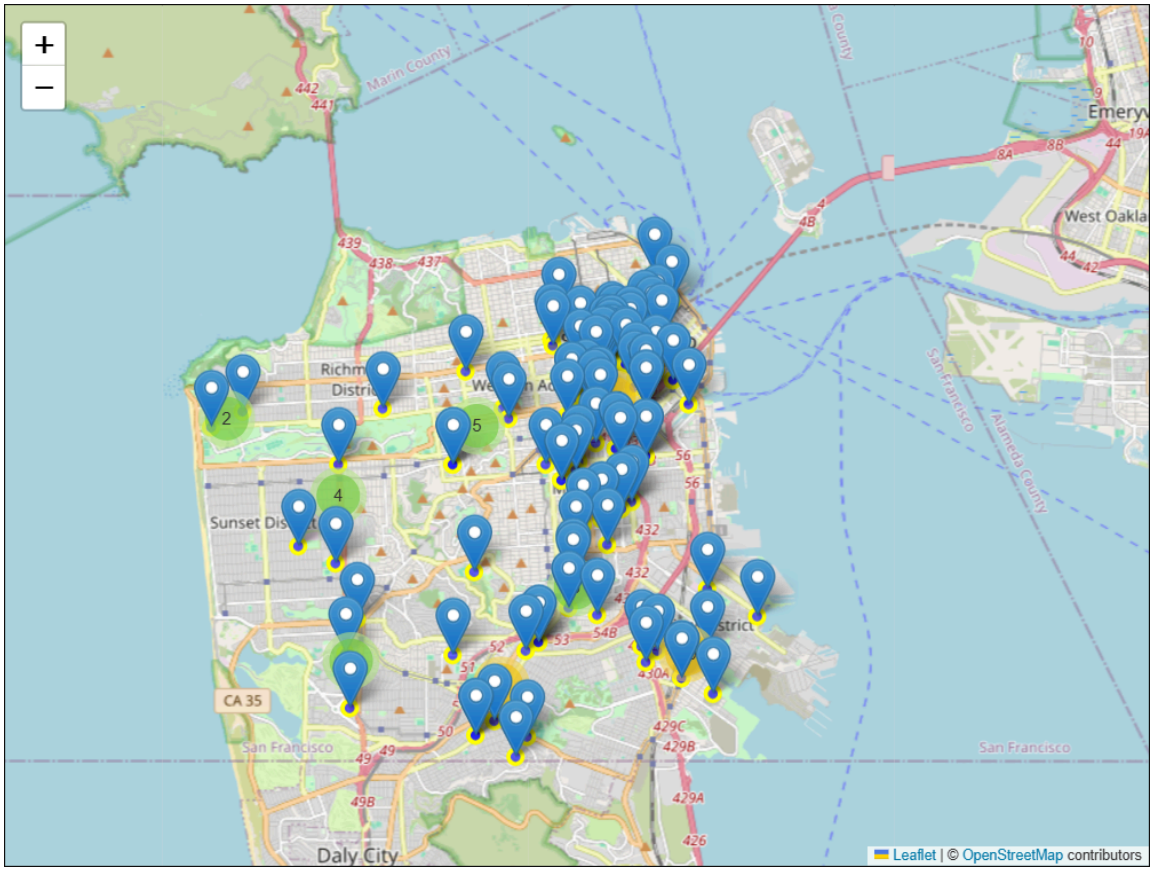
In [15]:
```
# instantiate a feature group for the incidents in the dataframe
"""incidents = folium.map.FeatureGroup()"""

# loop through the 100 crimes and add each to the incidents feature group
"""for lat, lng, in zip(df_incidents.Y, df_incidents.X):
    incidents.add_child(
        folium.vector_layers.CircleMarker(
            [lat, lng],
            radius=5, # define how big you want the circle markers to be
            color='yellow',
            fill=True,
            fill_color='blue',
            fill_opacity=0.6
        )
    )"""

# add incidents to map
"""sanfran_map.add_child(incidents)"""
```

Out[15]:
```
'sanfran_map.add_child(incidents)'
```

You can also add some pop-up text that would get displayed when you hover over a marker. Let's make each marker display the category of the crime when hovered over.

In [16]:
```
# map.FeatureGroup() object
"""incidents = folium.map.FeatureGroup(name="Suç Olayları")"""


"""for lat, lng, label in zip(df_incidents.Y, df_incidents.X, df_incidents.Category
    # 1. Circlemarker
"""    folium.CircleMarker(
        location=[lat, lng],
        radius=5,
                color='yellow',
                fill=True,
                fill_color='blue',
                fill_opacity=0.6
    ).add_to(incidents)"""

    # 2. Marker
"""    folium.Marker(
        location=[lat, lng],
        popup=label
    ).add_to(incidents)"""

# 4. add to the map
"""sanfran_map.add_child(incidents)

sanfran_map"""
```

Out[16]:
```
'sanfran_map.add_child(incidents)\n\nsanfran_map'
```



Isn't this really cool? Now you are able to know what crime category occurred at each marker.

If you find the map to be so congested will all these markers, there are two remedies to this problem. The simpler solution is to remove these location markers and just add the text to the circle markers themselves as follows:

In [17]:
```
# create map and display it
"""fig7 = Figure(800, 400)
sanfran_map = folium.Map(location=[latitude, longitude], zoom_start=12)"""

# loop through the 100 crimes and add each to the map
"""for lat, lng, label in zip(df_incidents.Y, df_incidents.X, df_incidents.Category
    folium.vector_layers.CircleMarker(
        [lat, lng],
        radius=5, # define how big you want the circle markers to be
        color='yellow',
        fill=True,
        popup=label,
        fill_color='blue',
        fill_opacity=0.6
    ).add_to(sanfran_map)
"""
# show map
"""fig7.add_child(sanfran_map)"""
```

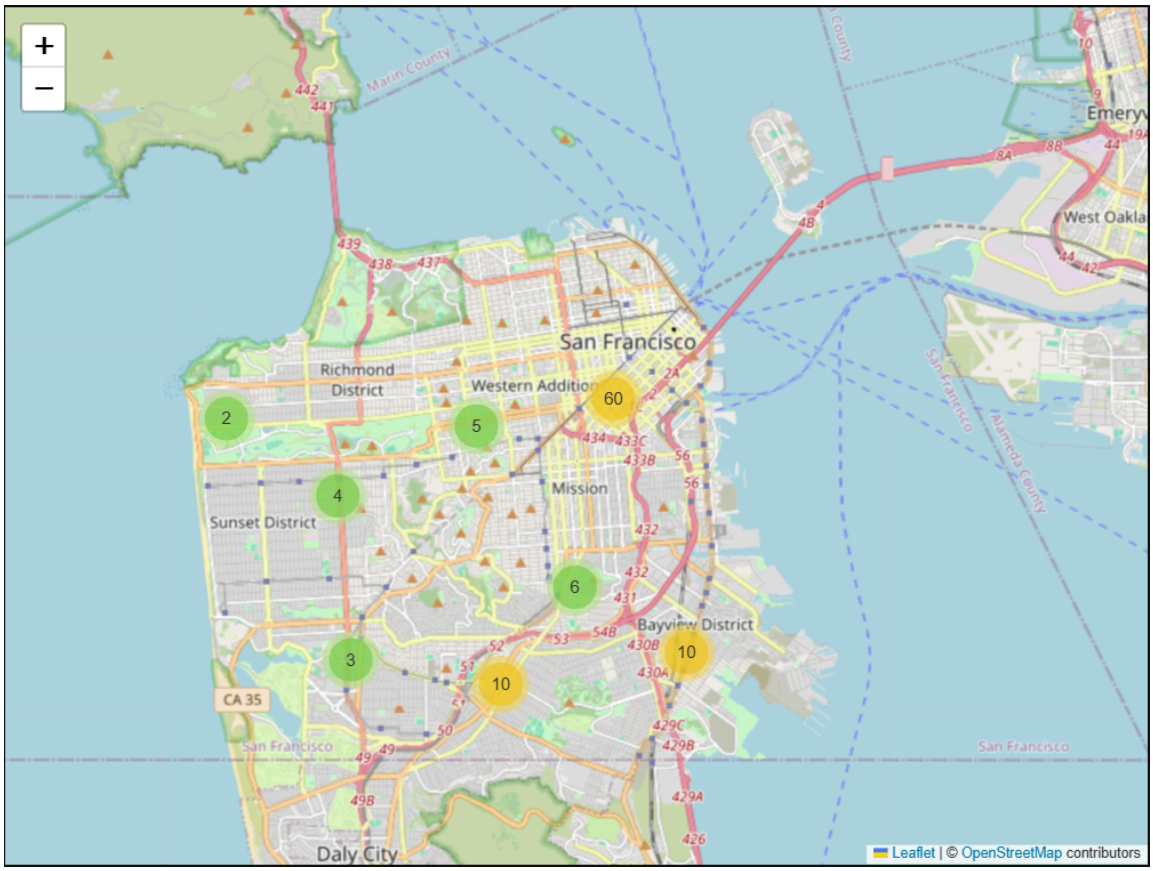
Out[17]:
```
'fig7.add_child(sanfran_map)'
```

The other proper remedy is to group the markers into different clusters. Each cluster is then represented by the number of crimes in each neighborhood. These clusters can be thought of as pockets of San Francisco which you can then analyze separately.

To implement this, we start off by instantiating a *MarkerCluster* object and adding all the data points in the dataframe to this object.

In [18]:
```
from folium import plugins

"""fig8 = Figure(800, 600)"""
# let's start again with a clean copy of the map of San Francisco
"""sanfran_map = folium.Map(location = [latitude, longitude], zoom_start = 12)"""

# instantiate a mark cluster object for the incidents in the dataframe
"""incidents = plugins.MarkerCluster()"""

# loop through the dataframe and add each data point to the mark cluster
"""for lat, lng, label, in zip(df_incidents.Y, df_incidents.X, df_incidents.Categor
    folium.Marker(
        location=[lat, lng],
        icon=None,
        popup=label,
    ).add_to(incidents)"""

# display map
"""sanfran_map.add_child(incidents)
fig8.add_child(sanfran_map)"""
```

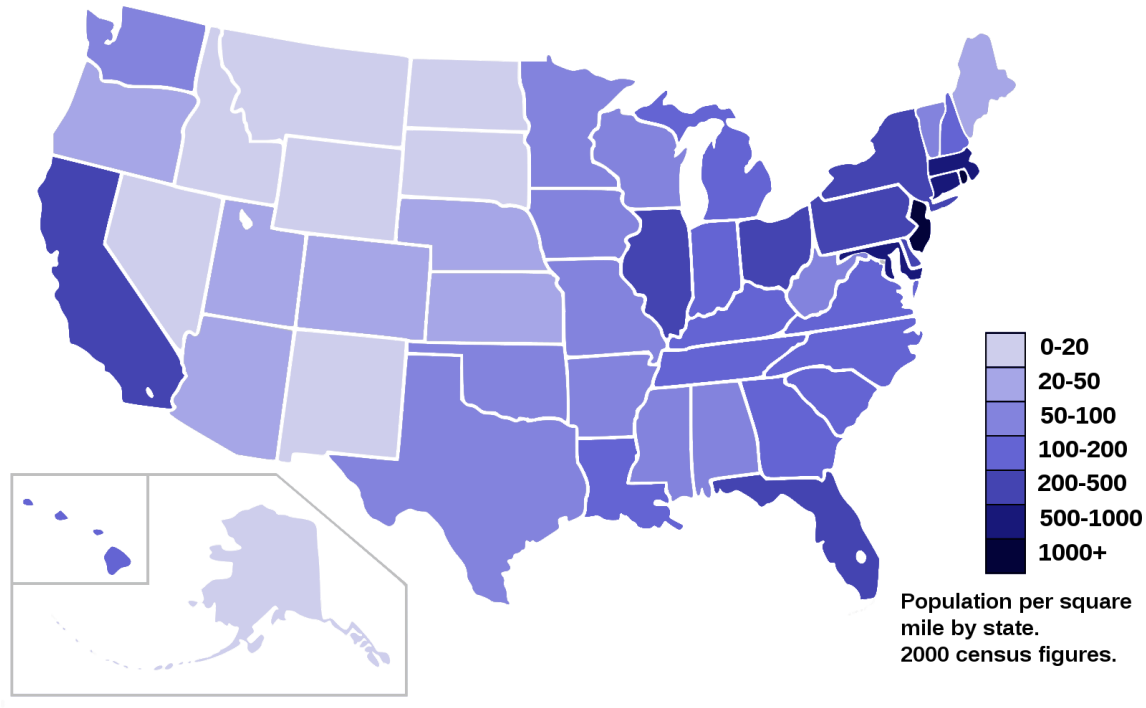
Out[18]:
```
'sanfran_map.add_child(incidents)\nfig8.add_child(sanfran_map)'
```



Notice how when you zoom out all the way, all markers are grouped into one cluster, *the global cluster*, of 100 markers or crimes, which is the total number of crimes in our dataframe. Once you start zooming in, the *global cluster* will start breaking up into smaller clusters. Zooming in all the way will result in individual markers.

Choropleth Maps

A `Choropleth` map is a thematic map in which areas are shaded or patterned in proportion to the measurement of the statistical variable being displayed on the map, such as population density or per-capita income. The choropleth map provides an easy way to visualize how a measurement varies across a geographic area, or it shows the level of variability within a region. Below is a `Choropleth` map of the US depicting the population by square mile per state.



Now, let's create our own `Choropleth` map of the world depicting immigration from various countries to Canada.

Download the Canadian Immigration dataset and read it into a *pandas* dataframe.

In [19]:
```
df_can = pd.read_csv('Canada.csv')
```

Let's take a look at the first five items in our dataset.

In [20]:
```
df_can.head()
```

Out[20]:

	Country	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	...
0	Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	...
1	Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	...
2	Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	...
3	American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	...
4	Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	...

5 rows × 39 columns

Let's find out how many entries there are in our dataset.

In [21]:
```
# print the dimensions of the dataframe
print(df_can.shape)
```

```
(195, 39)
```

In order to create a `Choropleth` map, we need a GeoJSON file that defines the areas/boundaries of the state, county, or country that we are interested in. In our case, since we are endeavoring to create a world map, we want a GeoJSON that defines the boundaries of all world countries. For your convenience, we will be providing you with this file, so let's go ahead and download it. Let's name it **world_countries.json**.

In [22]:
```
from urllib import request
import requests

url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDevelo

# file_name = "world_countries.json"
# request.urlretrieve(url, file_name)

geo_data = requests.get(url).text
```

Now that we have the GeoJSON file, let's create a world map, centered around **[0, 0]** *latitude* and *longitude* values, with an initisal zoom level of 2.

In [23]:
```
# create a plain world map
world_map = folium.Map(location=[0, 0], zoom_start=2)
```

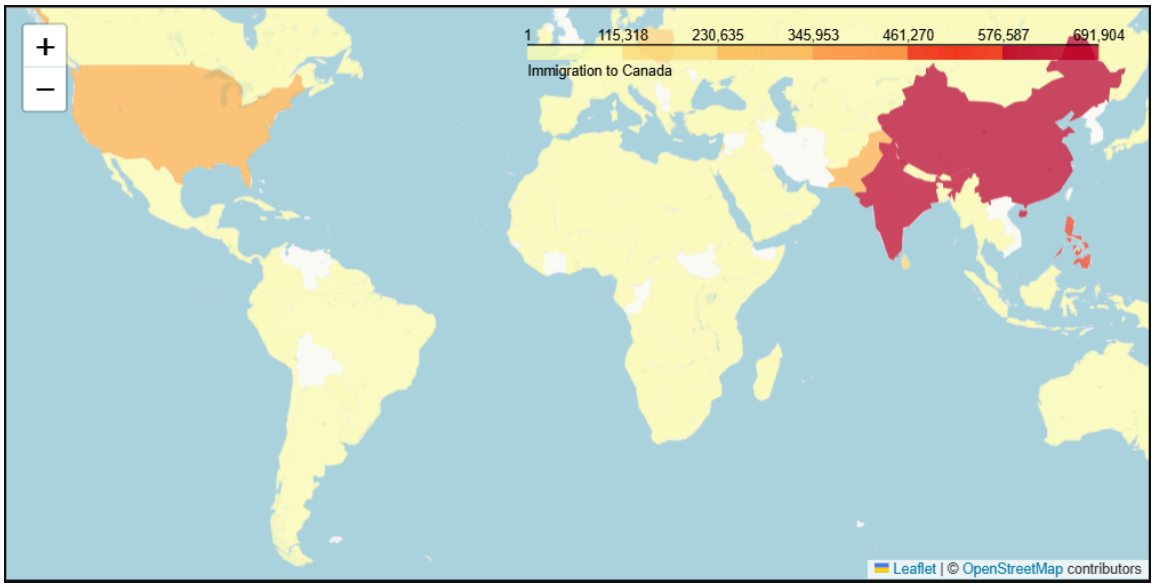
And now to create a `Choropleth` map, we will use the *choropleth* method with the following main parameters:

1. `geo_data` , which is the GeoJSON file.
2. `data` , which is the dataframe containing the data.
3. `columns` , which represents the columns in the dataframe that will be used to create the `Choropleth` map.
4. `key_on` , which is the key or variable in the GeoJSON file that contains the name of the variable of interest. To determine that, you will need to open the GeoJSON file using any text editor and note the name of the key or variable that contains the name of the countries, since the countries are our variable of interest. In this case, **name** is the key in the GeoJSON file that contains the name of the countries. Note that this key is case_sensitive, so you need to pass exactly as it exists in the GeoJSON file.

In [24]:
```
# generate choropleth map using the total immigration of each country to Canada fro
"""fig9 = Figure(800, 400)
world_map = folium.Map(location=[0, 0], zoom_start=2)
folium.Choropleth(
    geo_data=geo_data,
    data=df_can,
```

```
    columns=['Country', 'Total'],
    key_on='feature.properties.name',
    fill_color='YlOrRd',
    fill_opacity=0.7,
        nan_fill_color="white",
        line_color="b",
        line_weight=10,
        highlight=True,
    line_opacity=1,
    legend_name='Immigration to Canada',
    reset=True
).add_to(world_map)"""

# display map
"""world_map.add_to(fig9)"""
```

Out[24]: 'world_map.add_to(fig9)'



As per our `Choropleth` map legend, the darker the color of a country and the closer the color to red, the higher the number of immigrants from that country. Accordingly, the highest immigration over the course of 33 years (from 1980 to 2013) was from China, India, and the Philippines, followed by Poland, Pakistan, and interestingly, the US.

Feel free to play around with the data and perhaps create `Choropleth` maps for individuals years, or perhaps decades, and see how they compare with the entire period from 1980 to 2013.

Thank you for completing this lab!

Author

Alex Aklson.

##

© IBM Corporation 2020. All rights reserved.